Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java/Python

Pr. *Ilyass OUAZZANI TAYBI*

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026

Plan du cours

CHAPITRE 1 : INTRODUCTION À JAVA

1. Un peu d'histoire
2. Compilateur et Interpréteur
3. Conventions de nommage
4. Environnement de travail Java
5. Syntaxe du langage Java

I. Un peu d’histoire

Générations de langages de programmation

Génération	Exemples	Description
1ère génération	Langage machine (binaire)	Instructions directement comprises par le processeur
2ème génération	Assembleur	Langage symbolique bas-niveau
3ème génération	C, C++, Java, Python, Pascal…	Langages de haut niveau, lisibles par l'humain
4ème génération	SQL, MATLAB, R, SAS…	Langages déclaratifs pour applications spécifiques
5ème génération	Prolog, Mercury (IA, logique)	Langages basés sur la logique ou les contraintes, souvent utilisés en intelligence artificielle

I. Un peu d’histoire

Langage Java

- Le langage Java est né en **1991**, au sein d’une équipe de Sun Microsystems, dans un projet secret appelé "Green Project".
- Il a été conçu par **James Gosling**, qui l’a d’abord appelé « **Oak** », en référence à un chêne visible depuis la fenêtre de son bureau.
- **Objectif initial** : étudier l’impact de la convergence entre les **systèmes embarqués** et les ordinateurs personnels (**PC**).
- Avec l’essor du protocole HTTP et du Web, James Gosling a participé, en 1995, à la démonstration d’un navigateur capable d’afficher du contenu HTML enrichi avec des **applets Java**.

I. Un peu d’histoire

Langage Java

- L’évolution du langage a ensuite été prise en charge par la division **JavaSoft** (filiale de Sun Microsystems).
- Le langage Java a été officiellement annoncé en **1995** avec une version bêta du **JDK** (**Java Development Kit**), tandis que la première version publique et stable, le **JDK 1.0**, est sortie en **1996**.
- La version la plus récente est **Java SE 23** (**2024**)

I. Un peu d’histoire

Langage Java

Trois environnements d’exécution de Java:

- **Java SE (Standard Edition)** : destiné principalement aux applications de bureau (desktop). C’est l’édition de base qui fournit le langage, le JDK et les bibliothèques fondamentales.
- **Java EE (Enterprise Edition)** : conçu pour les applications serveur et d’entreprise. Il repose sur Java SE mais ajoute d’autres technologies (Servlet, JSP, EJB, …).
- **Java ME (Micro Edition)** : destiné aux appareils mobiles et embarqués (PDA, téléphones, cartes SIM, etc.). C’est une version allégée de Java SE, avec certains paquetages en moins pour s’adapter aux contraintes matérielles.

II. Compilateur et Interpréteur

```
public abstract class classAstratta {
    public abstract void AbstractMethod1();
        //Metodo astratto di tipo Void
    public abstract String AbstractMethod2();
        //Metodo astratto di tipo String
    public abstract int AbstractMethod3();
        //Metodo astratto di tipo Int
    public abstract double AbstractMethod4();
        //Metodo astratto di tipo Double
}
```

Langage haut niveau

Traduction

Langage machine

Les **Compilateurs** et les **Interpréteurs** sont des traducteurs de langage de programmation de haut niveau en séries d'instructions-machine directement exécutables par l'ordinateur.

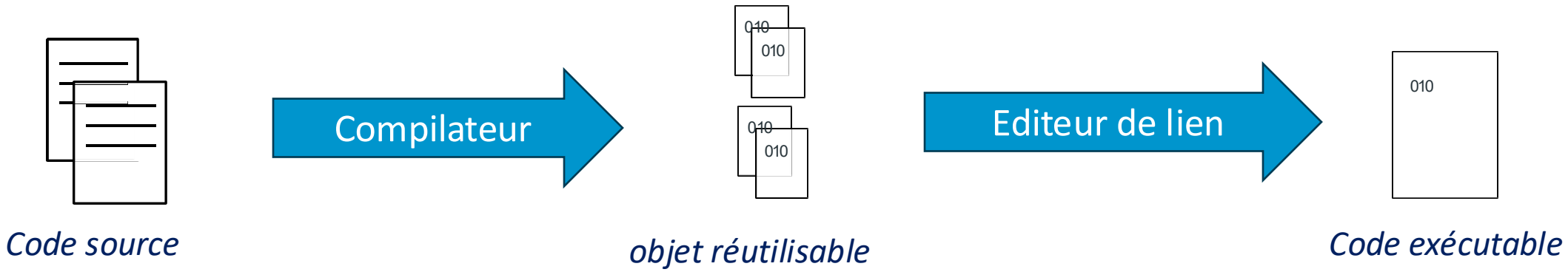
II. Compilateur et Interpréteur

Compilateur

- Le **compilateur** est un programme qui **traduit l'intégralité du code source en une seule fois**, généralement vers un langage intermédiaire (**bytecode**) ou directement en **code machine** exécutable par l'ordinateur.
- **Avantages** :
  - Exécution plus rapide car le code est déjà traduit en langage machine,
  - Détection des erreurs de syntaxe avant l'exécution.
- **Inconvénients** :
  - Un simple changement dans le code nécessite une recompilation complète,
  - Temps de compilation parfois long.
  - Le code machine généré dépend de l'architecture matérielle, ce qui limite la portabilité
- Exemples : C, C++, Java (compilé en bytecode puis exécuté par la JVM).

II. Compilateur et Interpréteur

Compilateur



- Lors de la compilation, chaque fichier source (.c, .cpp, etc.) est compilé séparément.
- Cela produit un fichier objet par fichier source (.o sous Linux, .obj sous Windows).
- Ensuite, l'éditeur de liens (linker) regroupe tous ces fichiers objets + les bibliothèques pour générer un seul executable.

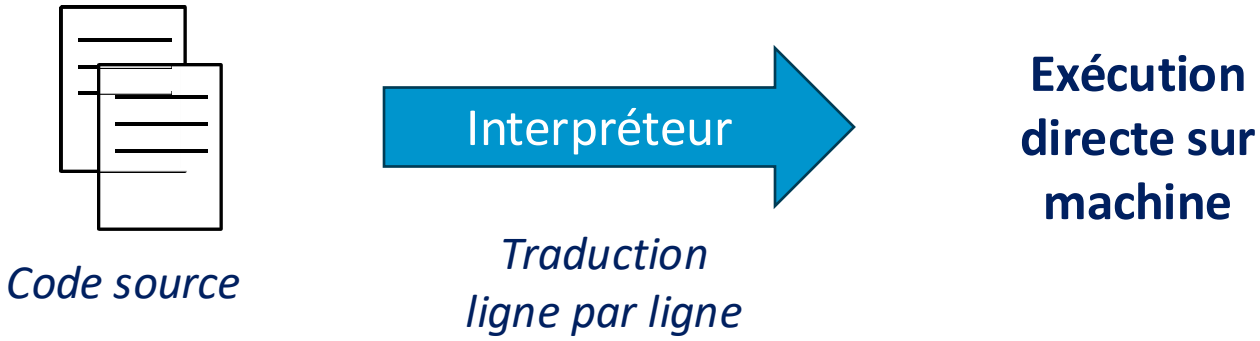
II. Compilateur et Interpréteur

Interpréteur

- L'**interpréteur** est un programme qui **traduit et exécute le code source ligne par ligne ou instruction par instruction**, sans produire de fichier exécutable indépendant.
- **Avantages** :
  - Pas de phase de compilation préalable : exécution immédiate du programme.
  - Flexibilité : possibilité de tester et modifier rapidement le code.
  - Portabilité : le même code peut s'exécuter sur différentes plateformes, à condition que l'interpréteur soit disponible.
- **Inconvénients** :
  - Exécution plus lente, car le code est traduit à chaque lancement.
  - Erreurs détectées uniquement à l'exécution, pas en amont.
  - Nécessite la présence de l'interpréteur sur la machine cible.
- Exemples : Python, JavaScript, Ruby, PHP, Java (partiellement interprété : bytecode exécuté par la JVM).

II. Compilateur et Interpréteur

Interpréteur



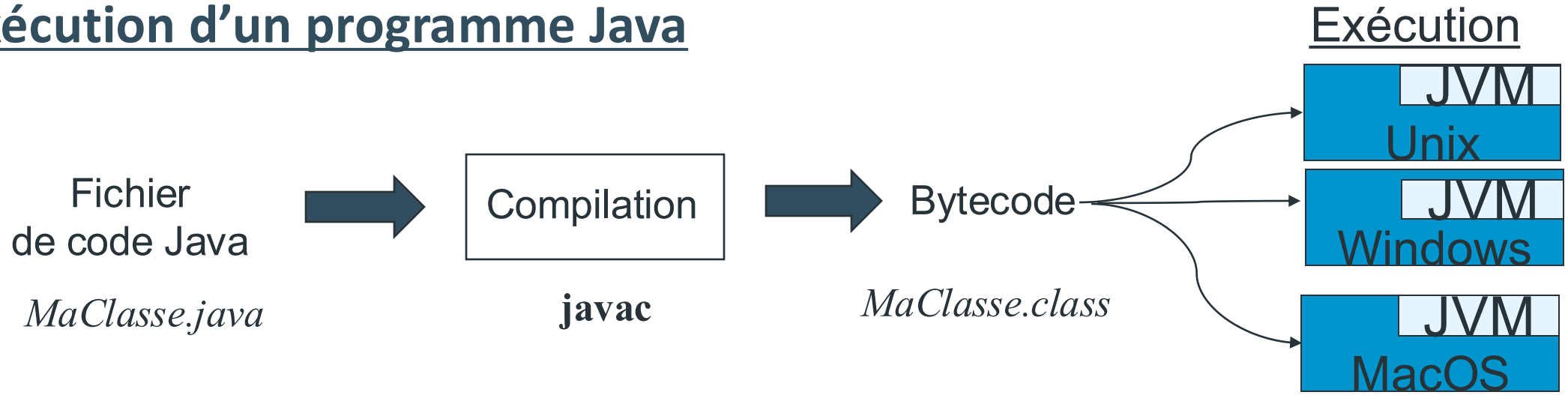
- Le code est exécuté directement par l'interpréteur, instruction par instruction.
- Pas de fichier exécutable indépendant généré.
- L'exécution est plus lente mais plus flexible et portable.

II. Compilateur et Interpréteur

Catégorie	Définition	Caractéristiques	Exemples
Langages compilés	Le **code source** est **entièrement traduit en code machine natif** par un compilateur avant exécution.	- Production d'un exécutable indépendant. - Exécution très rapide. - Dépendance à l'architecture matérielle. - Temps de compilation parfois long.	C, C++, Fortran, Pascal, Ada, COBOL
Langages interprétés	Le **code source** est **traduit et exécuté instruction par instruction** par un interpréteur.	- Grande flexibilité et interactivité. - Plus lents à l'exécution. - Pas d'exécutable natif autonome. - Idéal pour le scripting, le web et le prototypage rapide.	Python, JavaScript, Ruby, PHP
Langages hybrides	Combinaison **compilation + interprétation** : le **code source** est **compilé** en **bytecode** intermédiaire, puis **exécuté** par une **machine virtuelle** ou un **interpréteur**.	- Portabilité entre systèmes (Write Once, Run Anywhere). - Performances correctes (optimisations JIT). - Dépend d'une machine virtuelle (JVM, CLR).	Java (JVM), C# (.NET CLR), Kotlin, Scala

II. Compilateur et Interpréteur

Exécution d'un programme Java



- Le code source **.java** est compilé par **javac** en **bytecode** contenu dans un fichier **.class**.
- Ce **bytecode** est **interprété** ou **compilé à la volée** (**JIT**: Just In Time) par la Java Virtual Machine (JVM), qui se charge de l'exécution sur la machine hôte.

- *Le **bytecode** est un langage intermédiaire entre le code source écrit par le programmeur et le code machine compréhensible par le processeur.*

II. Compilateur et Interpréteur

Exécution d'un programme Java

Avantages :

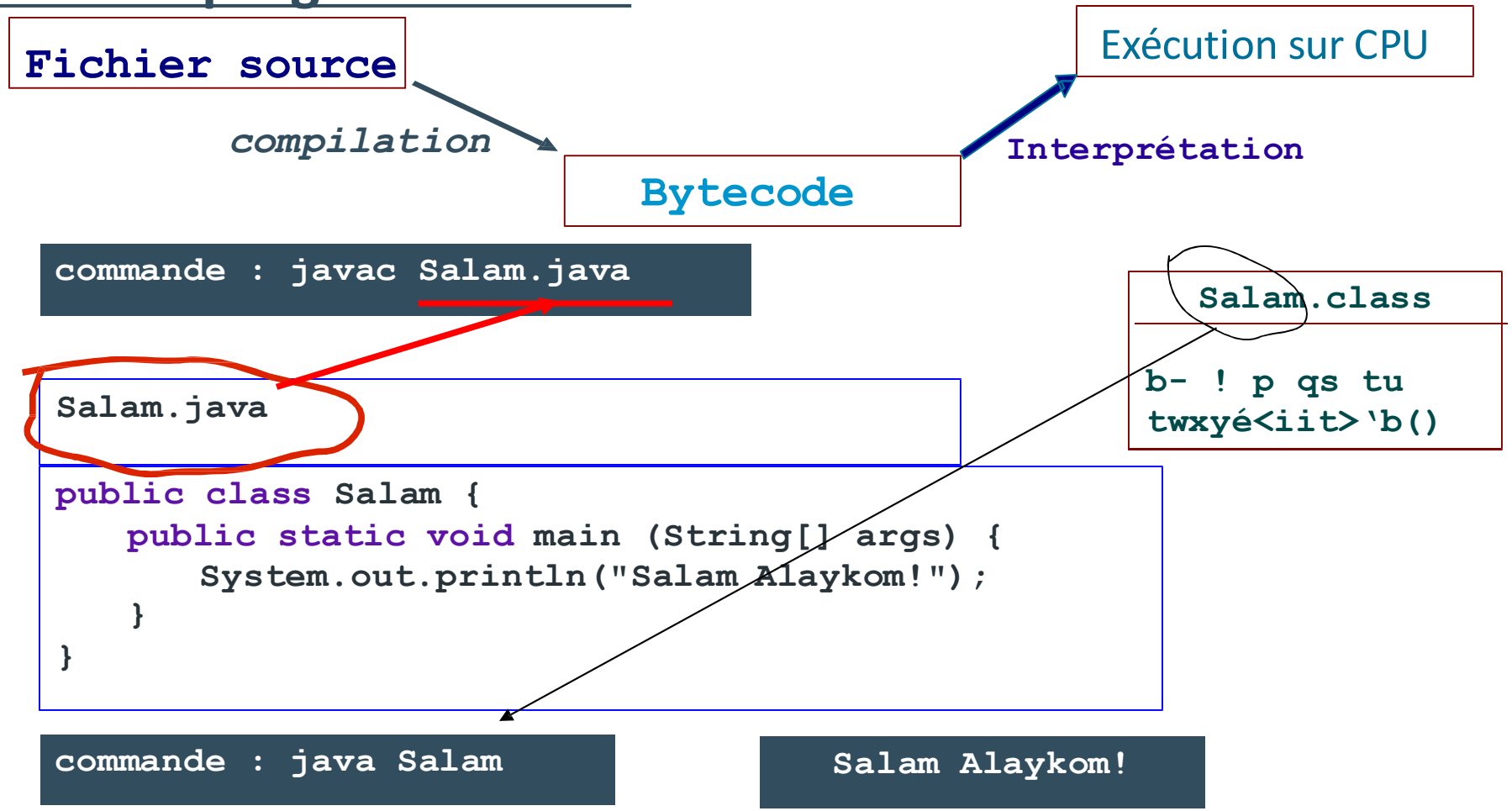
- **Portabilité totale** : le bytecode peut s'exécuter sur n'importe quel système disposant d'une JVM (**Write Once, Run Anywhere**).
- **Optimisations dynamiques** grâce au **compilateur JIT** (Just-In-Time), qui traduit les portions de code fréquemment utilisées en code machine natif.
- **Sécurité renforcée** : la JVM fournit une couche de protection.

Inconvénients :

- **Performance moindre** par rapport à un langage compilé natif (comme C/C++), car il y a une étape intermédiaire (bytecode + JVM).
- **Dépendance à la JVM** : nécessité d'avoir la machine virtuelle installée.
- **Temps de démarrage plus long qu'un exécutable natif**, car la JVM doit être initialisée.

II. Compilateur et Interpréteur

Exécution d'un programme Java



L'interpréteur Java s'appelle la machine virtuelle Java
(Java Virtual Machine (JVM))

III. Conventions de nommage

Conventions de nommage en JAVA

- Nom de classe/interface :
  - Commencer chaque mot par une majuscule (**PascalCase**)
    - Exemple : MaClasse, MonPremierProgramme
- Nom de variable ou méthode :
  - Commencer chaque mot par une majuscule, sauf le premier (**camelCase**)
    - Exemple : maPremiereVariable, var1, maPremiereMethode, main
- Constantes :
  - Tout en majuscules avec underscores:
    - Exemple : MA_CONSTANTE, PI

IV. Environnement de travail Java

Rôle du JDK

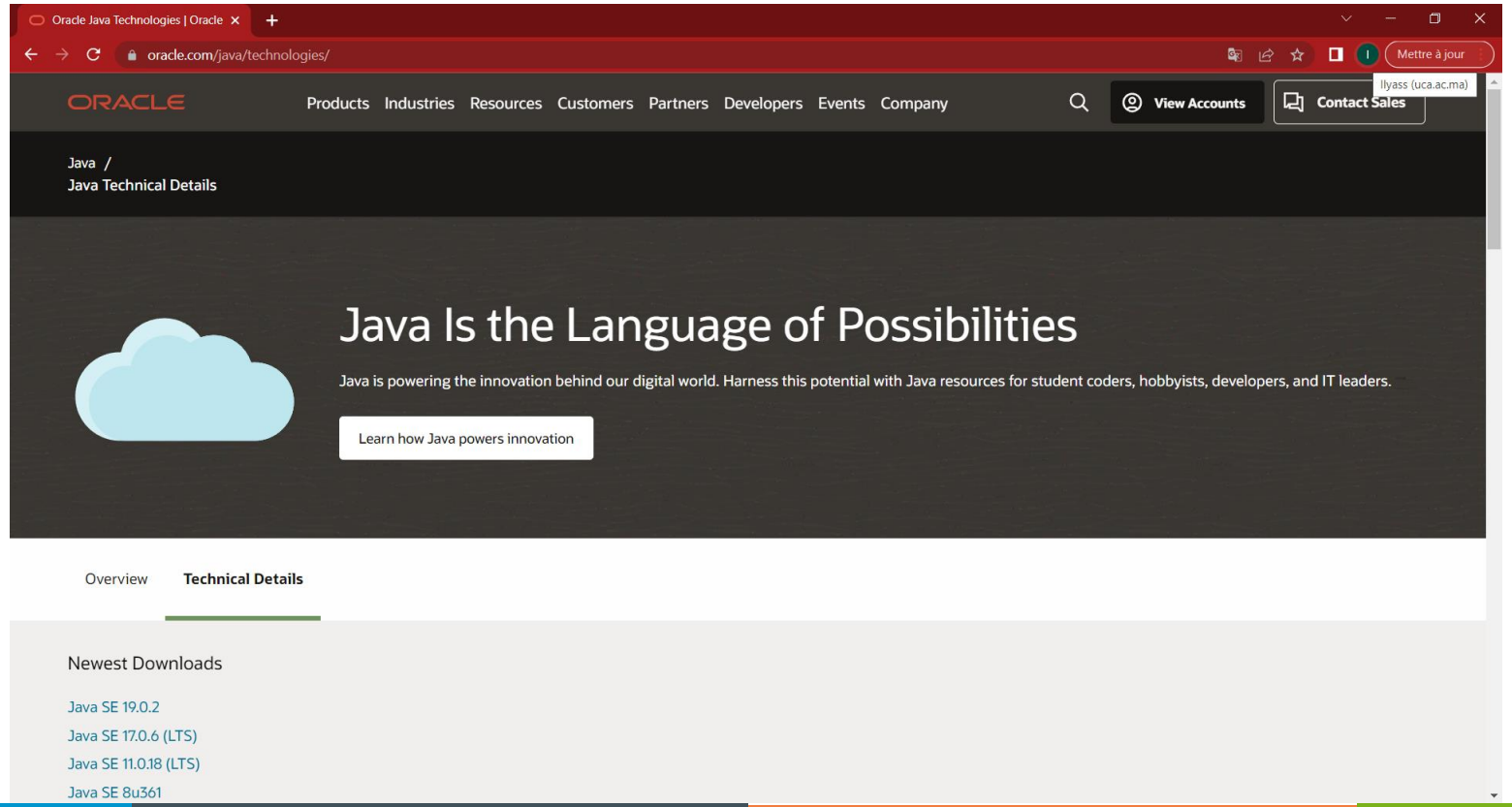
- Le **JDK** (Java Development Kit en anglais, Kit de Développement Java en Français) représente l'outillage indispensable au développeur Java.

- Ce kit contient les outils nécessaires pour programmer, exécuter, tester et livrer des programmes java.

IV. Environnement de travail Java

Téléchargement :

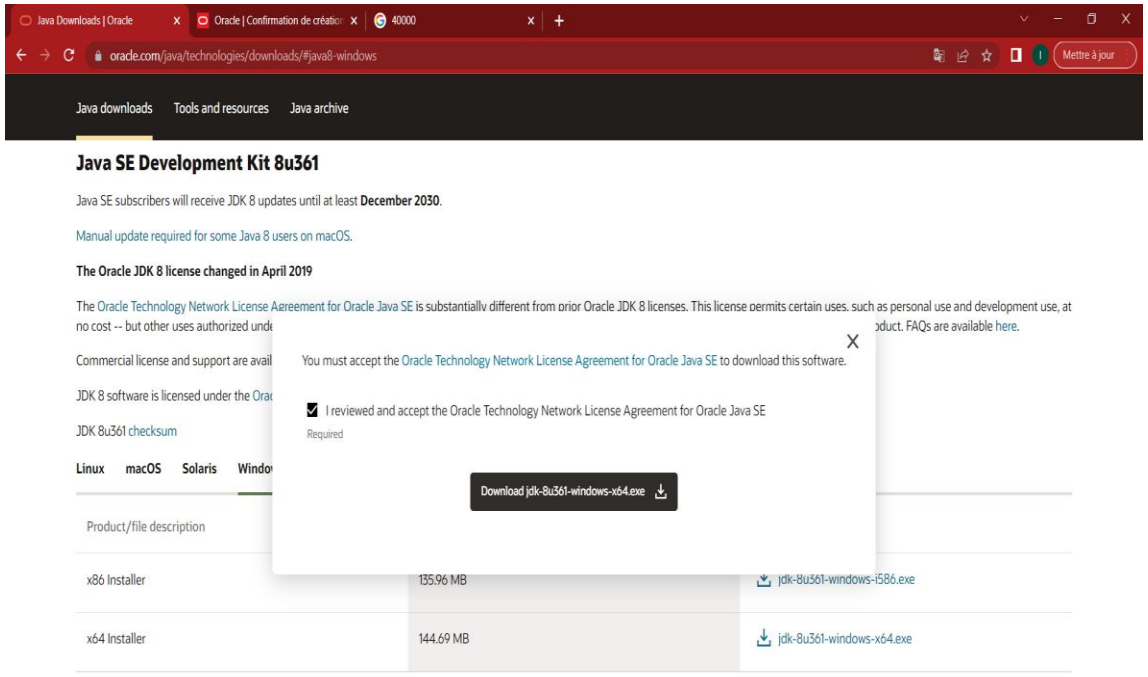
- Le **JDK** peut être téléchargé gratuitement à partir du site d'oracle sur (https://www.oracle.com/technetwork/java/).



IV. Environnement de travail Java

Installation du JDK:

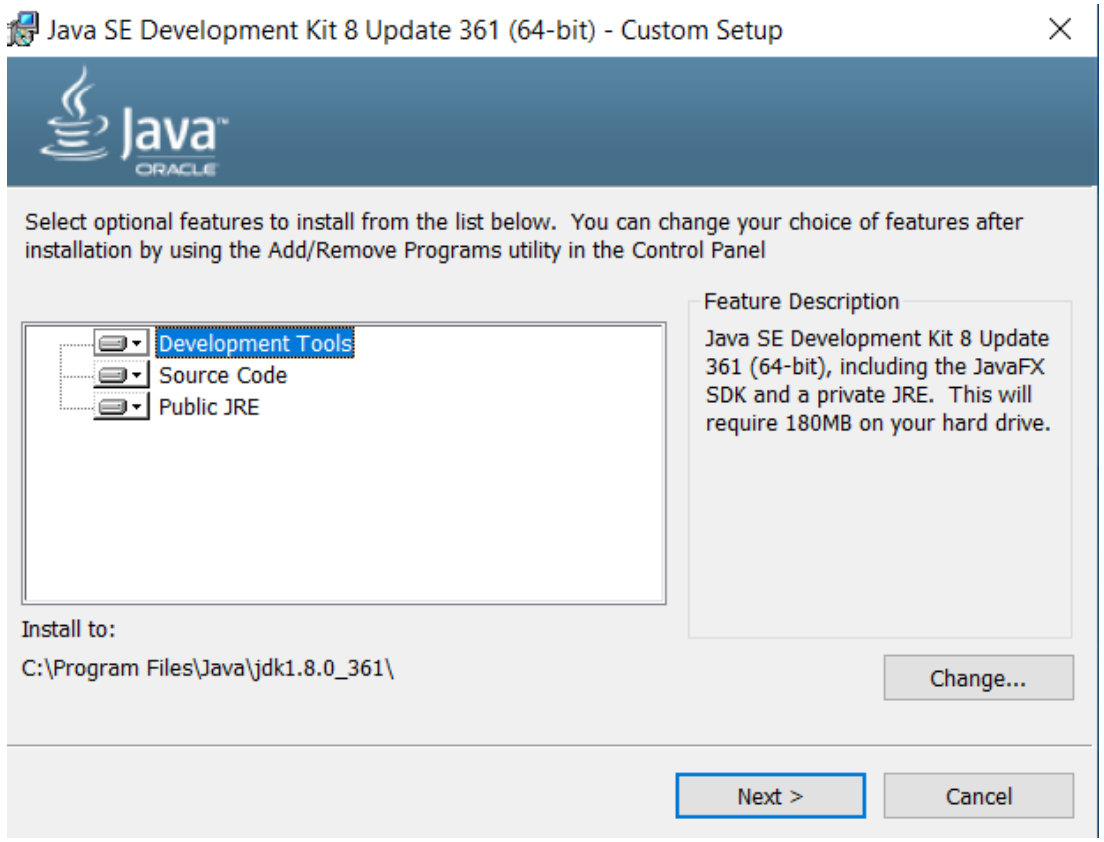
- Dans la zone ‘Newest Downloads’, cliquez sur le lien ‘Java SE 8u361’. L’écran suivant apparaît.
- Il faut Accepter la licence
- Choisissez le produit en fonction du system d’exploitation (Windows, Linux …) et en fonction du processeur de votre machine soit 64 bit ou 32 bit.
  - Si vous avez un Windows 32bits choisissez : jdk-8u361-windows-i586.exe
  - Si vous avez un Windows 64bits choisissez : jdk-8u361-windows-x64.exe



Installation du JDK:

- Cliquez sur le bouton ‘Next’ dessus. L’écran suivant (Licence) apparaît.

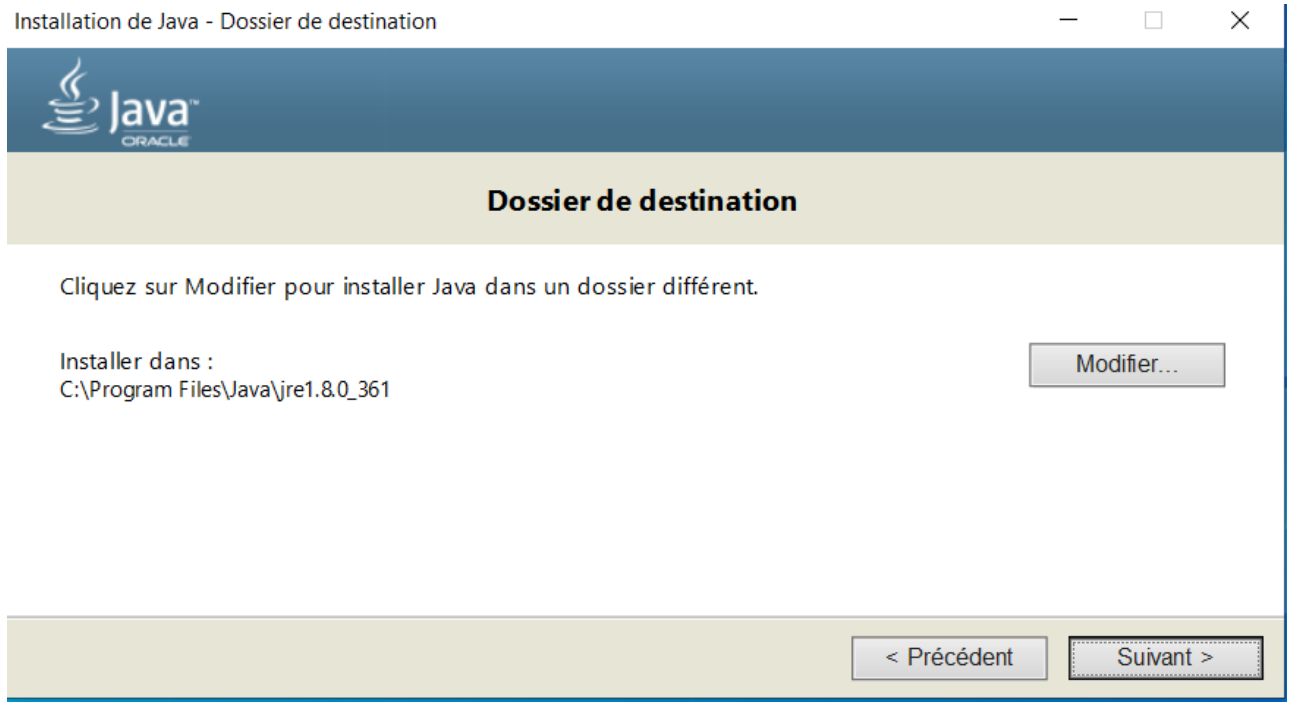
- Cet écran vous permet de choisir les modules à installer ou non dont le code source du JDK et l’environnement d’exécution Java (le JRE).



IV. Environnement de travail Java

Installation du JDK:

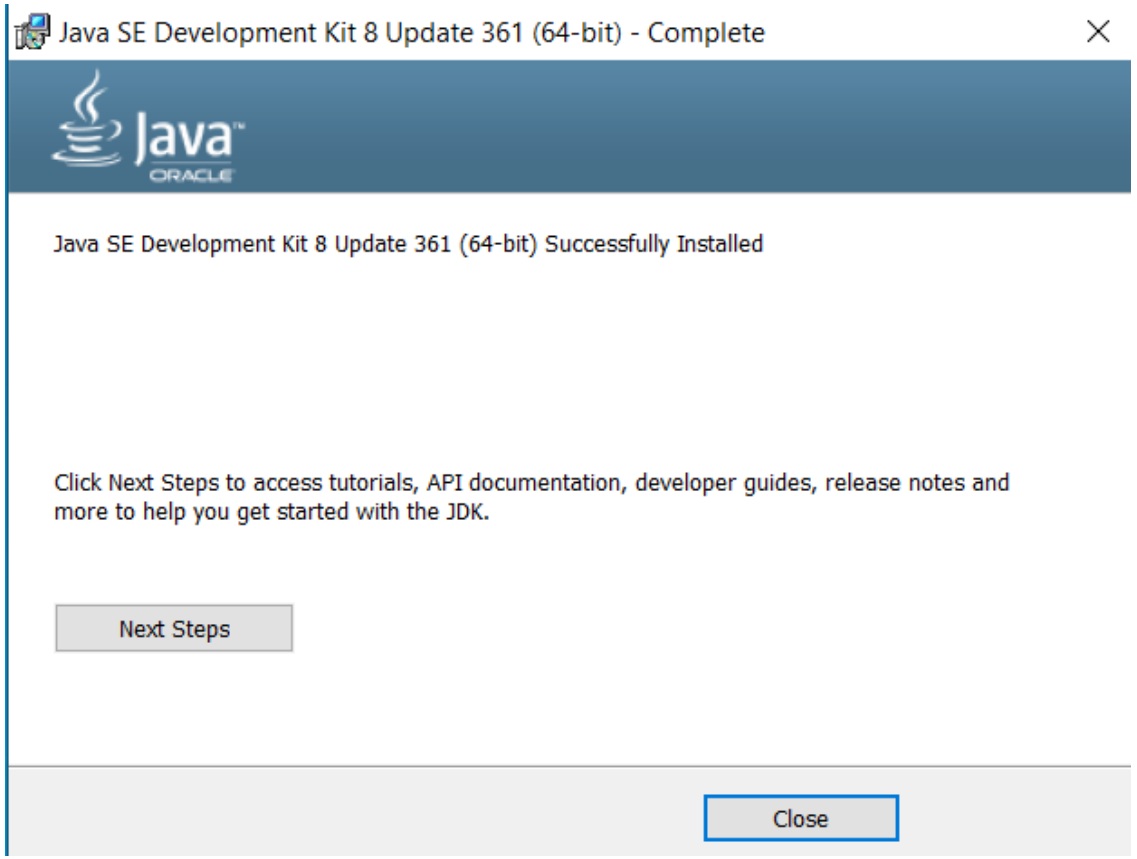
- L'assistant propose aussi un répertoire d'installation par défaut : « C:\Program Files\Java\jre1.8.0_361 »
- Cliquez sur 'Next' et l'installation se poursuit.



Installation du JDK:

- Vous arrivez à l'écran de fin d'installation, qui vous précise que l'installation s'est bien déroulée .

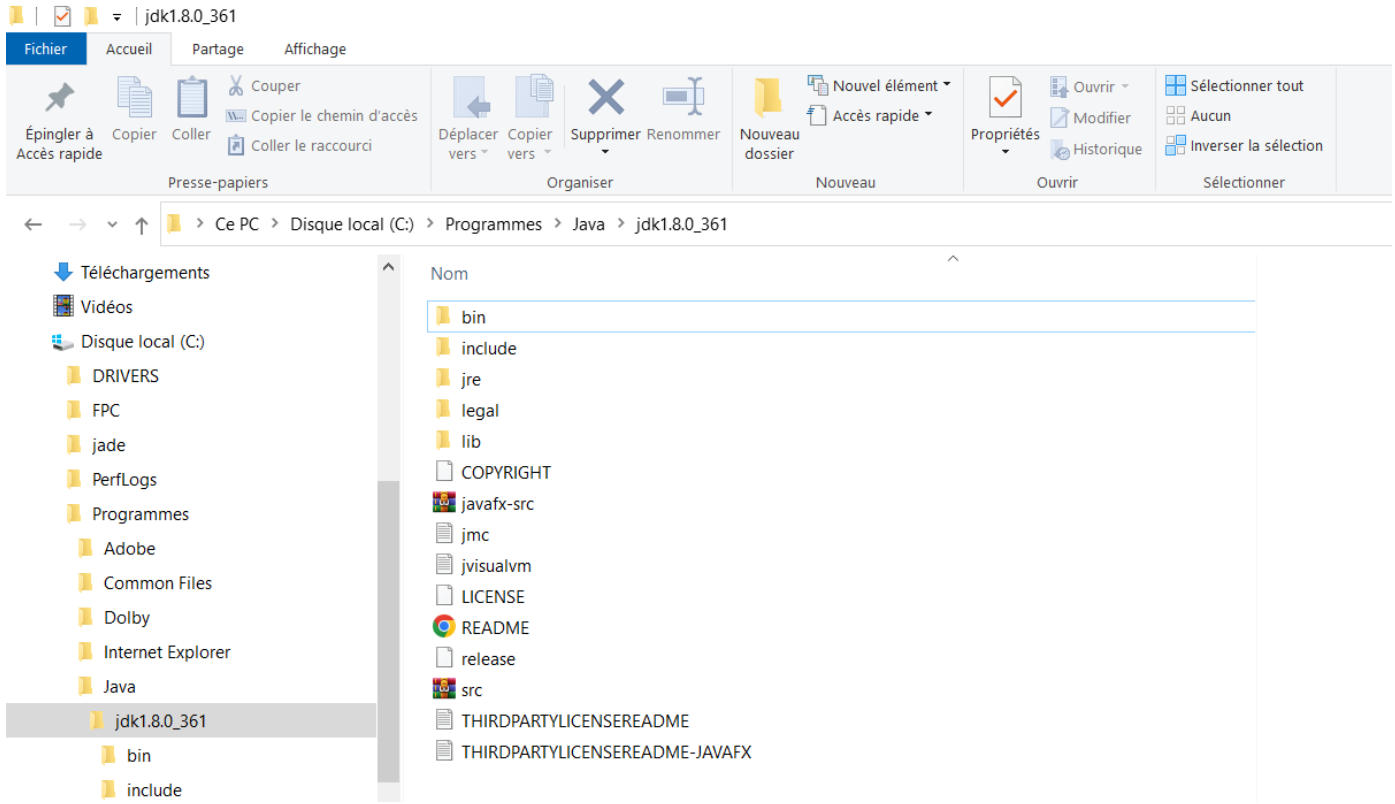
- Cliquez sur « Close » et l'installation du JDK est terminée.



IV. Environnement de travail Java

Analyse du contenu et des répertoires du JDK

- Vérifiez l'arborescence de votre installation, sous le répertoire de Java et plus précisément dans (C:\Program Files\Java\jdk1.8.0_361)

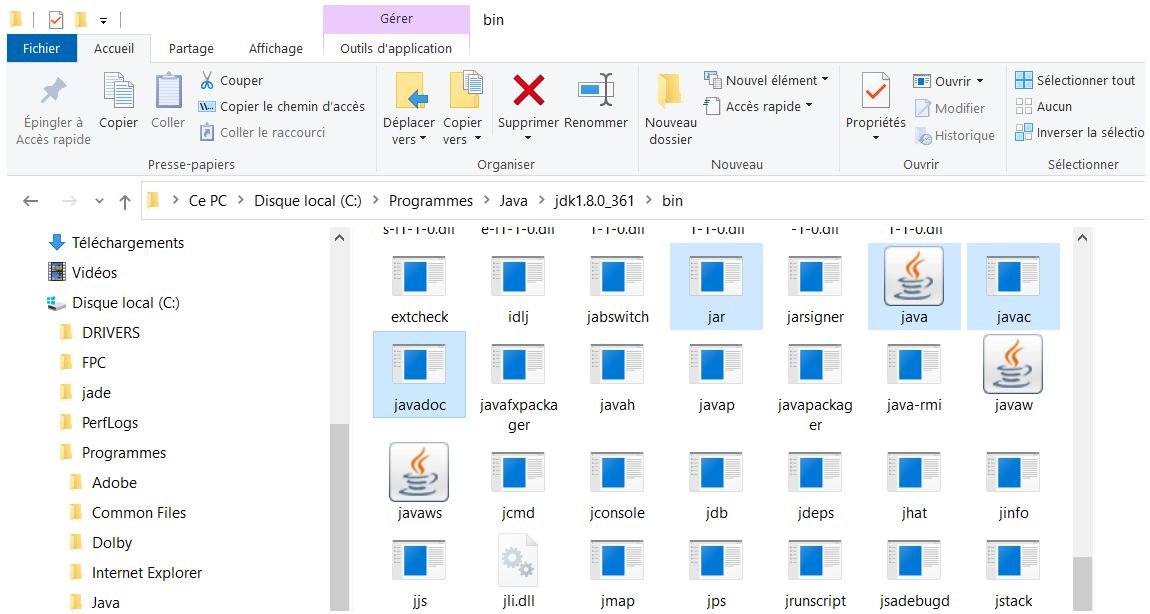


IV. Environnement de travail Java

Analyse du contenu et des répertoires du JDK

- Une fois sur le répertoire du JDK et spécialement sur le répertoire ‘bin’ qui contient tous les outils du JDK, on peut remarqué la présence d’un ensemble d’exécutables

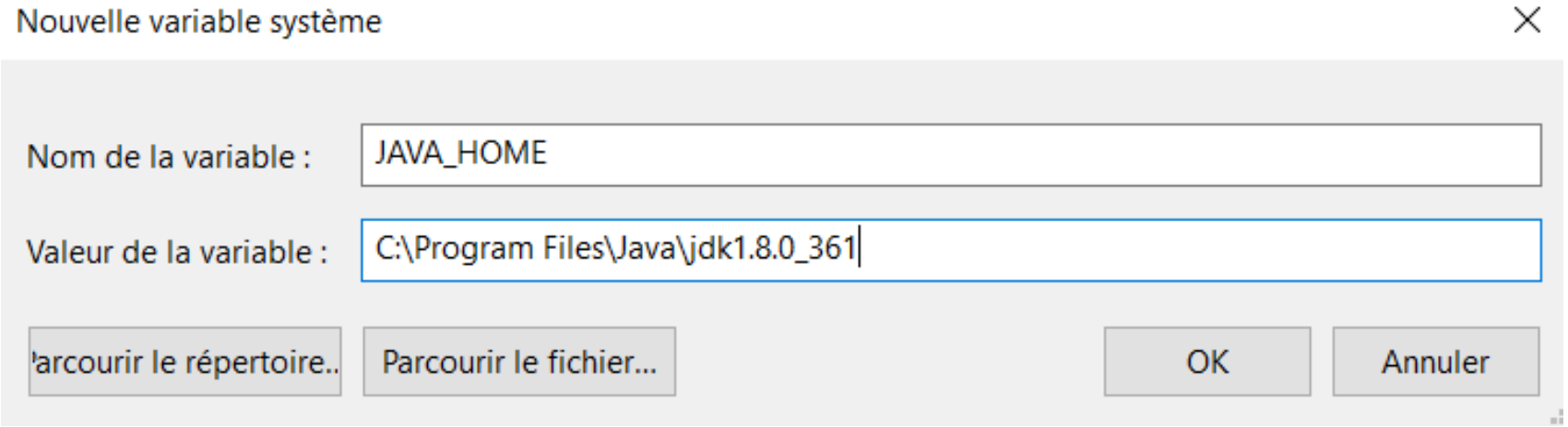
- Notez les 4 outils/fichiers exécutables mis en évidence. Ils constitueront les premiers outils que vous apprendrez à utiliser :
  - Le compilateur : **javac**
  - La machine virtuelle : **java**
  - L’outil de génération de documentation : **Javadoc**
  - L’outil de création de ‘livrables’ pour vos clients : **jar**



IV. Environnement de travail Java

JAVA_HOME

La variable JAVA_HOME est une variable d'environnement. Elle correspond au répertoire dans lequel est installé le kit de développement Java (JDK).



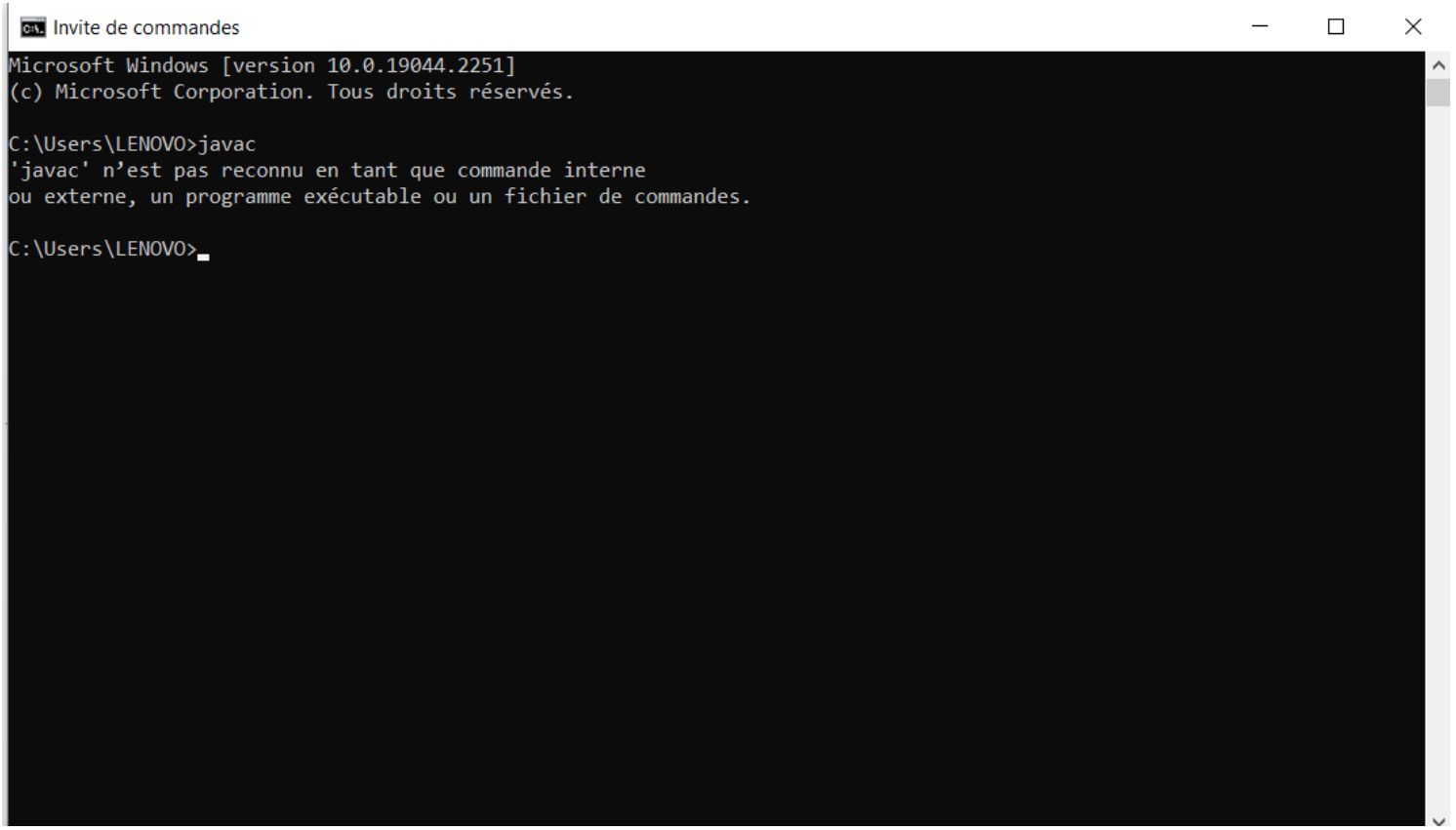
- Cherchez la variable JAVA_HOME dans la liste des variables systèmes. Si vous la trouvez, cliquez sur Modifier pour changer sa valeur. Sinon, cliquez sur Nouvelle pour la créer. Renseignez le chemin d'installation du JDK puis cliquez sur OK puis Appliquer pour valider la modification.

IV. Environnement de travail Java

Test de l'installation

- Ouvrez une invite de commande et tapez **javac**:

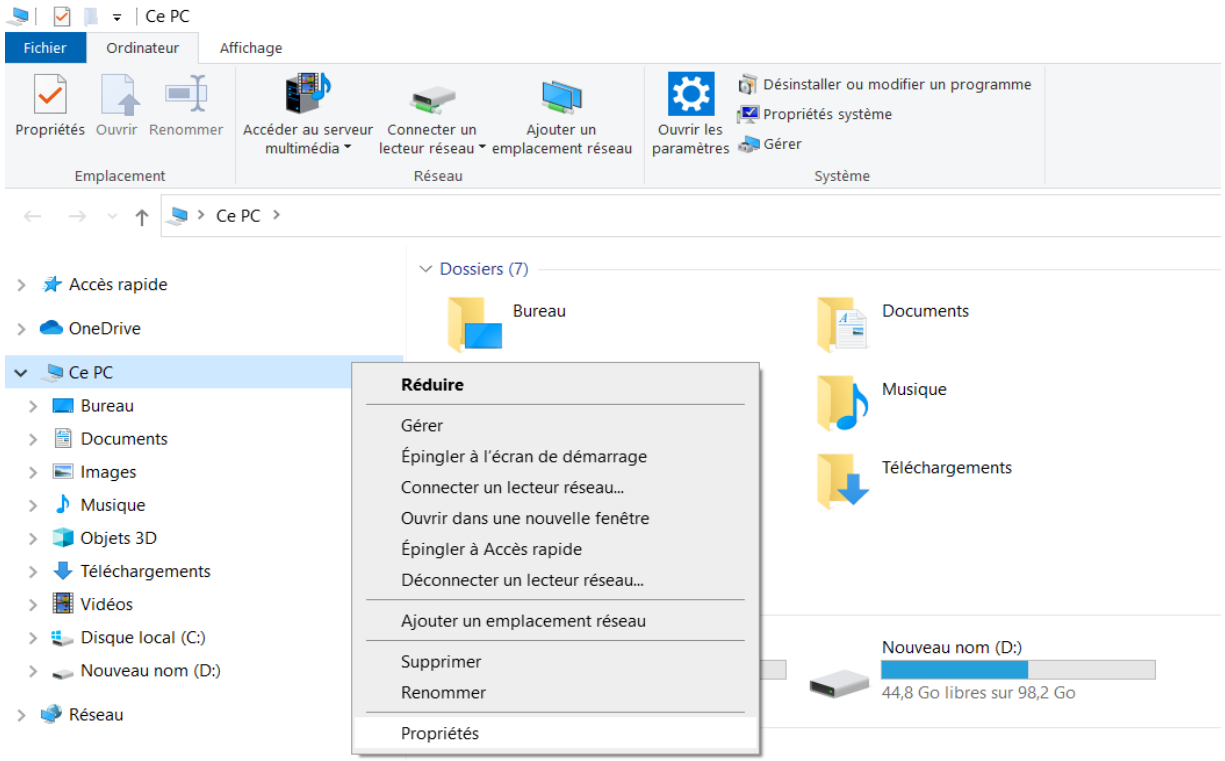
- Comme on peut le constater, le système ne reconnaît pas cette commande. Pour ce faire, et afin que le système reconnaisse TOUS les outils présents dans le répertoire 'bin', vous devez ajoutez ce répertoire à la variable d'environnement 'PATH' de votre ordinateur.



IV. Environnement de travail Java

Test de l'installation

- Pour modifier les variables d'environnement 'PATH' de votre ordinateur, vous devez sélectionnez 'propriétés' de votre 'Ordinateur'
- Et puis cliquez sur le lien 'paramètres systèmes avancés'.



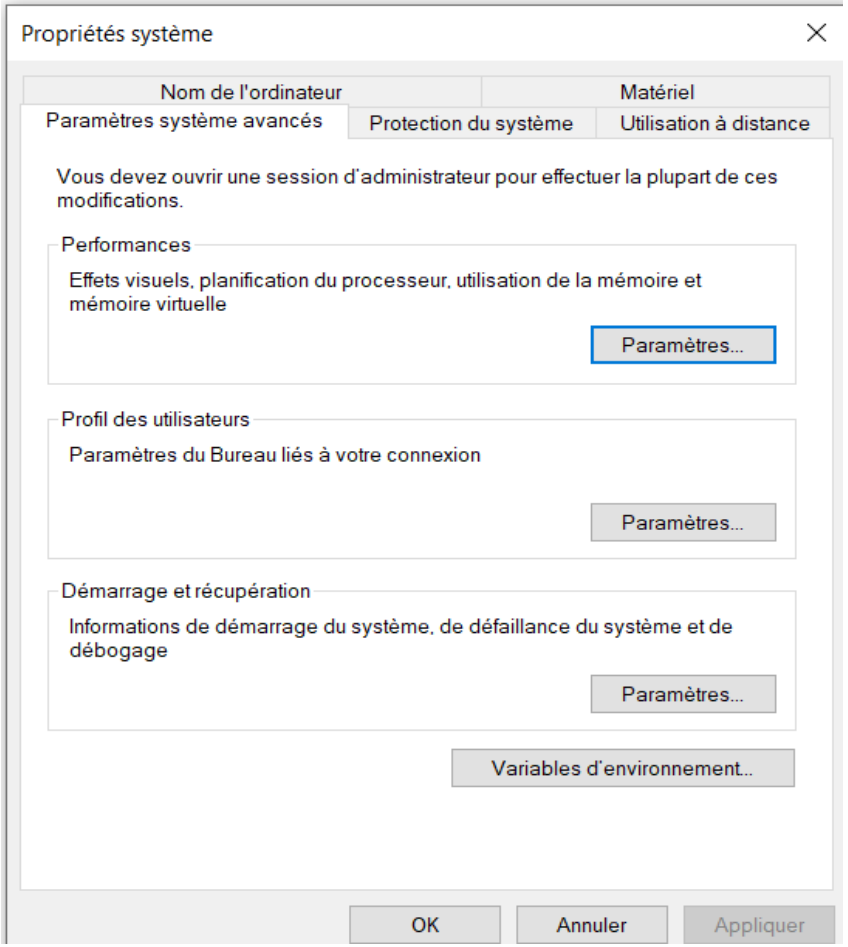
IV. Environnement de travail Java

Fin de l'installation

Pour modifier les variables d'environnement 'PATH' de votre ordinateur, vous devez sélectionnez 'propriétés' de votre 'Ordinateur':

- Cliquez sur le bouton 'variable d'environnement'. Vous obtenez la fenêtre des variables d'environnement, contenant une partie 'user' et une partie 'système'.
- Modifier la variable PATH en ajoutant la valeur suivante:
- C:\Program Files\Java\jdk1.8.0_361\bin; Ou %JAVA_HOME%\bin;

Une fois fait, toutes les commandes Java seront reconnues.



IV. Environnement de travail Java

Deux façons d'écrire des programmes Java :

- **En écrivant le code dans un simple éditeur de texte**
  - Compilation et exécution du code en ligne de commande DOS
  - Exemple d'éditeurs de textes:
    - Sublime Text (https://www.sublimetext.com/)
    - Atom (https://atom.io/)
    - Brackets (http://brackets.io/)
    - Notepad++ (https://notepad-plus-plus.org/fr/)
    - …
- **En utilisant un environnement de développement (IDE)**
  - Netbeans (http://www.netbeans.org)
  - Elipse (https://www.eclipse.org)
  - Visual Studio Code (+ extensions Java) (https://code.visualstudio.com/)
  - …

IV. Environnement de travail Java

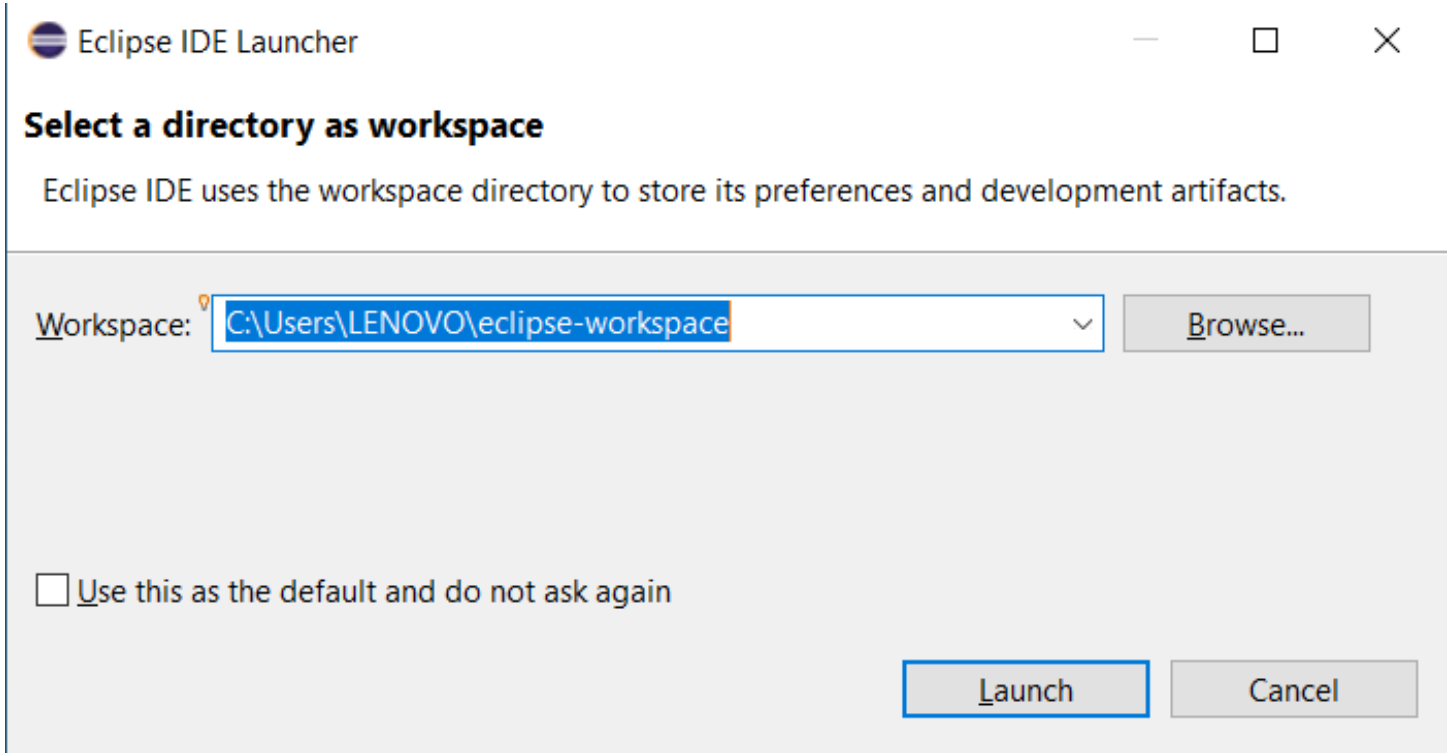
Eclipse

- Eclipse est très utilisé dans le monde académique et professionnel.
- Pour le lancer, il suffit de double-cliquer sur le fichier exécutable « eclipse.exe » présent dans le répertoire ‘eclipse’.

IV. Environnement de travail Java

Eclipse

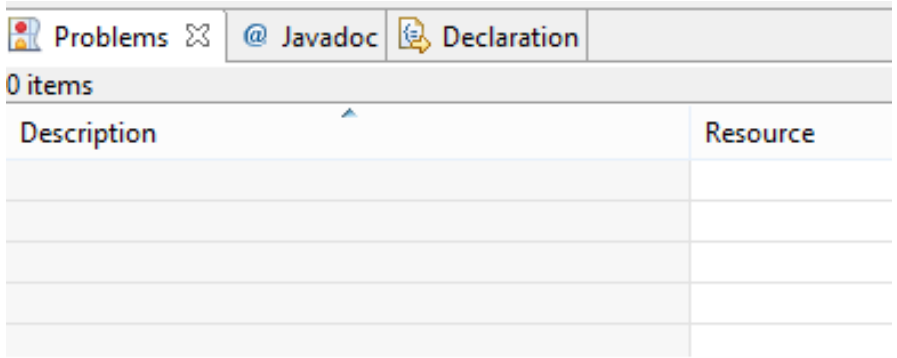
- Un écran apparaît ensuite pour vous demander le ‘**workspace**‘ (Répertoire de travail). C’est dans ce répertoire que Eclipse stockera les programmes (*.java) et fichiers de configuration (*.xml, *.properties, …) de vos projets.

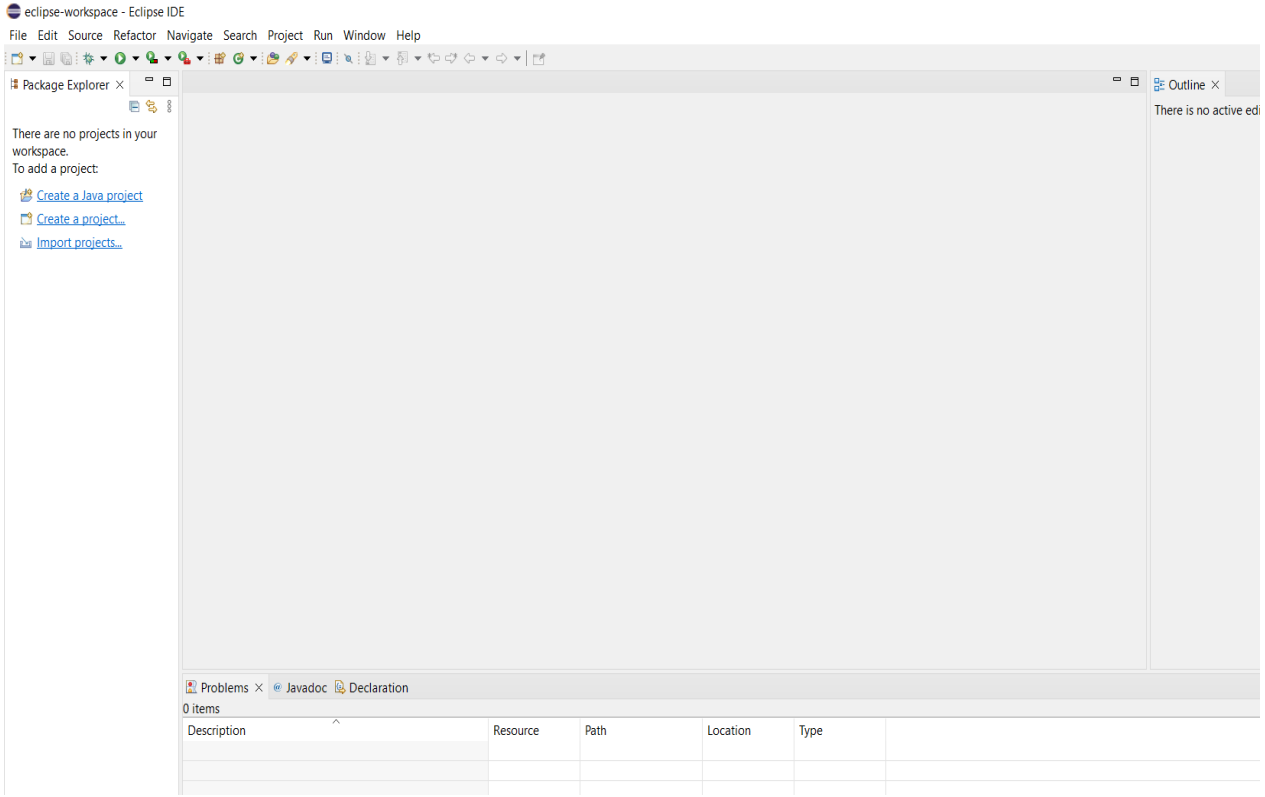


IV. Environnement de travail Java

Eclipse

- Une fois Eclipse est lancé, on peut voir un ensemble de vues, où une vue est une petite fenêtre possédant des informations particulières.
- Par exemple vous pouvez voir les vues ‘Problems’, ‘Javadoc’ et ‘Declaration’

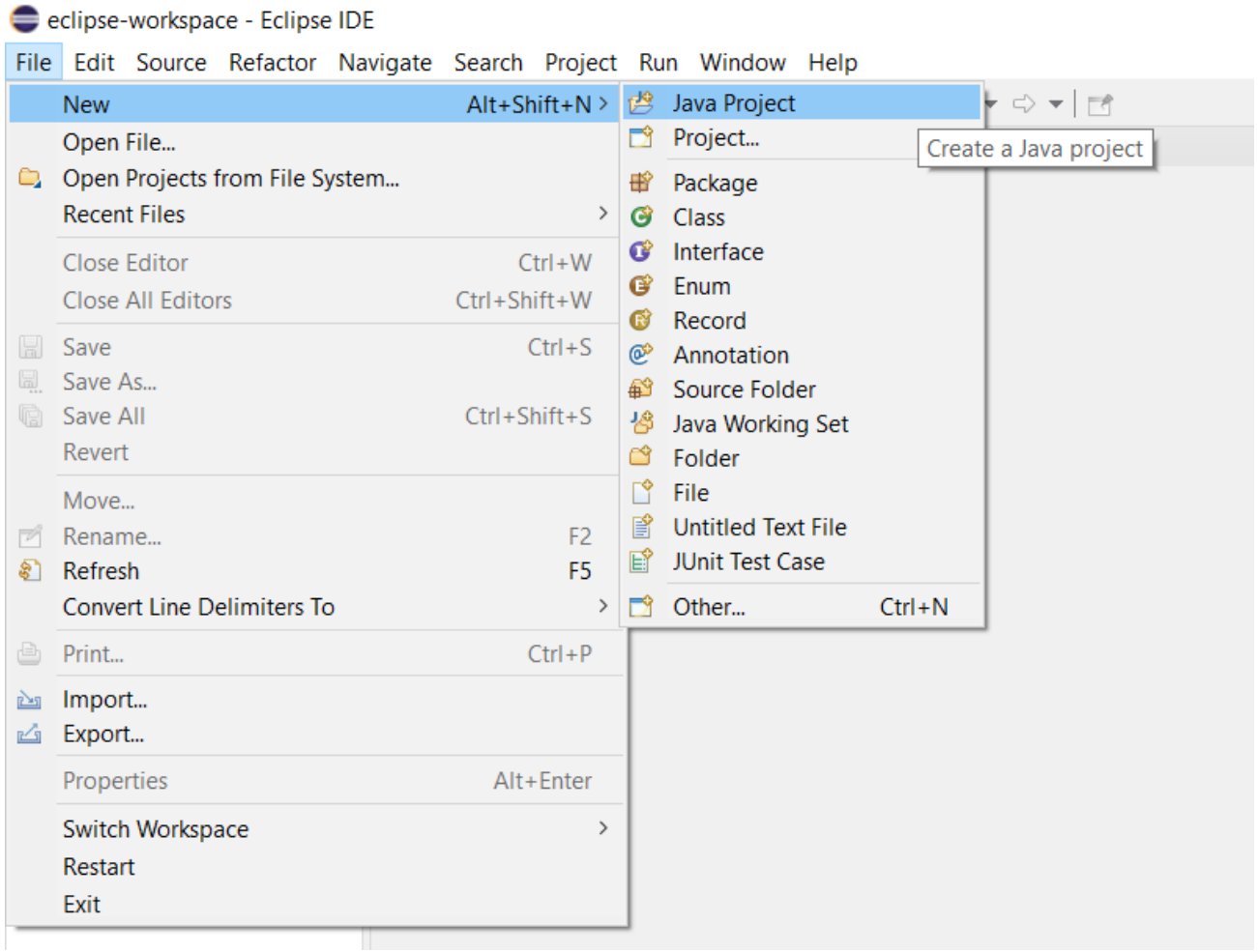




IV. Environnement de travail Java

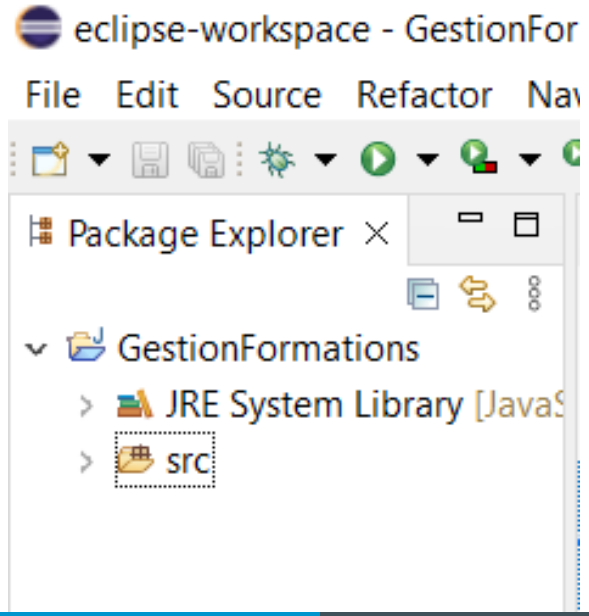
Création projet Java

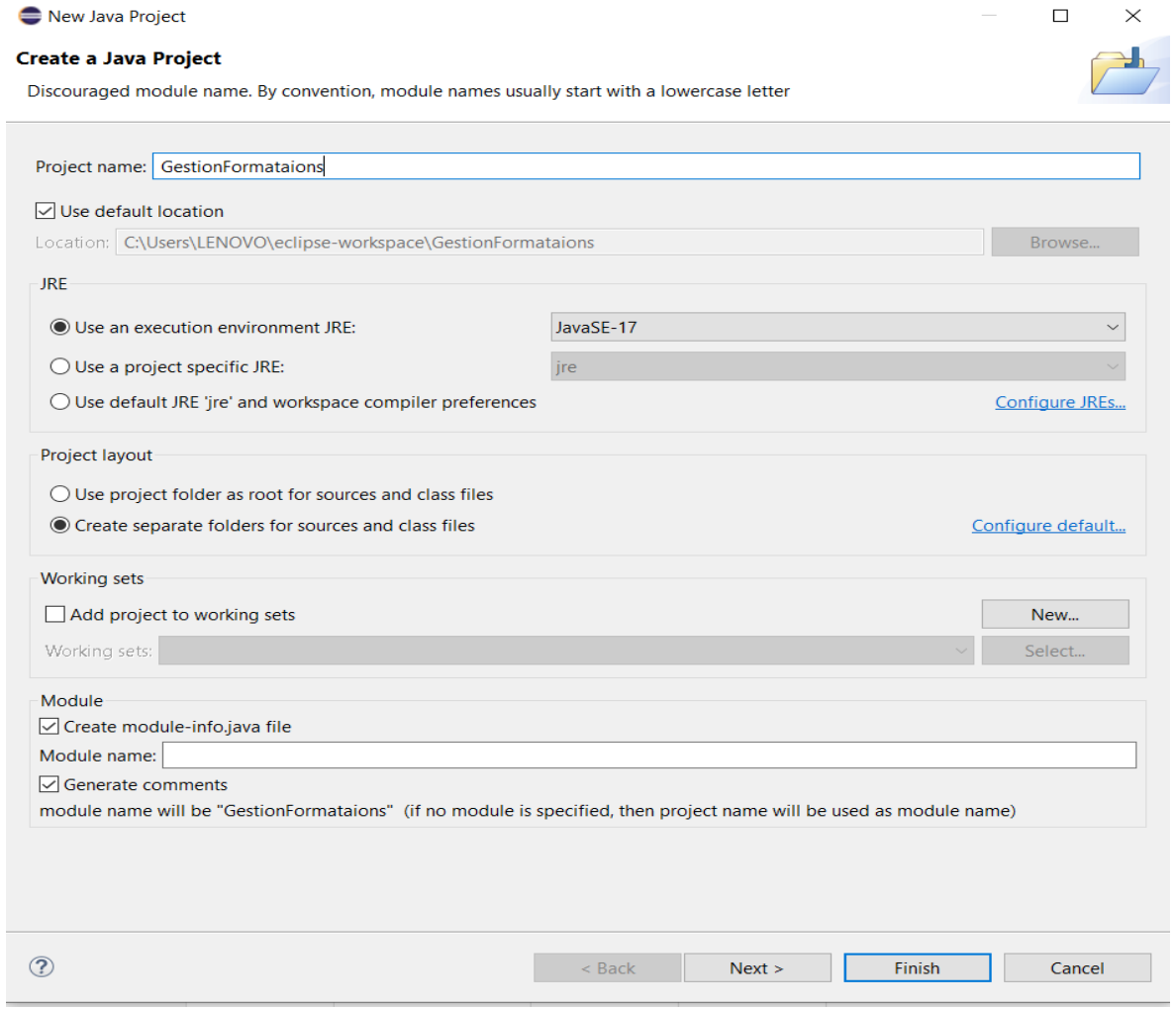
Menu File/new/Java Project



Création projet Java

- puis bouton ‘Next’. L’écran suivant s’affiche.
- Et en cliquant sur Finish, Eclipse crée alors le squelette du projet.

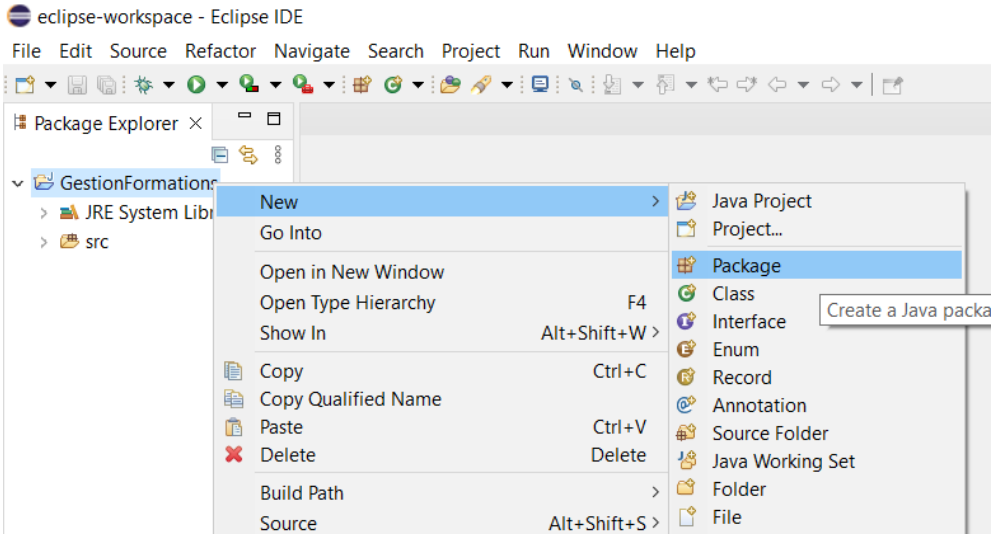


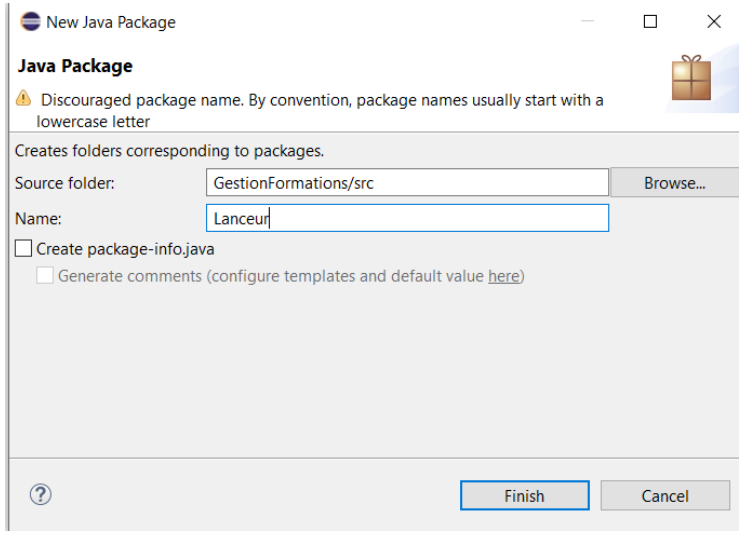


IV. Environnement de travail Java

Création projet Java

- Cliquez droit sur le projet (GestionFormations) dans la vue ‘project Explorer’ puis new/package
- Entrez ‘Lanceur’ comme nom de package. Puis cliquez sur le bouton Finish

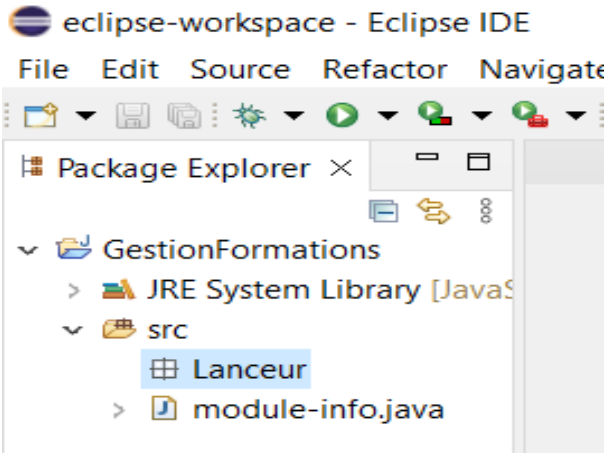


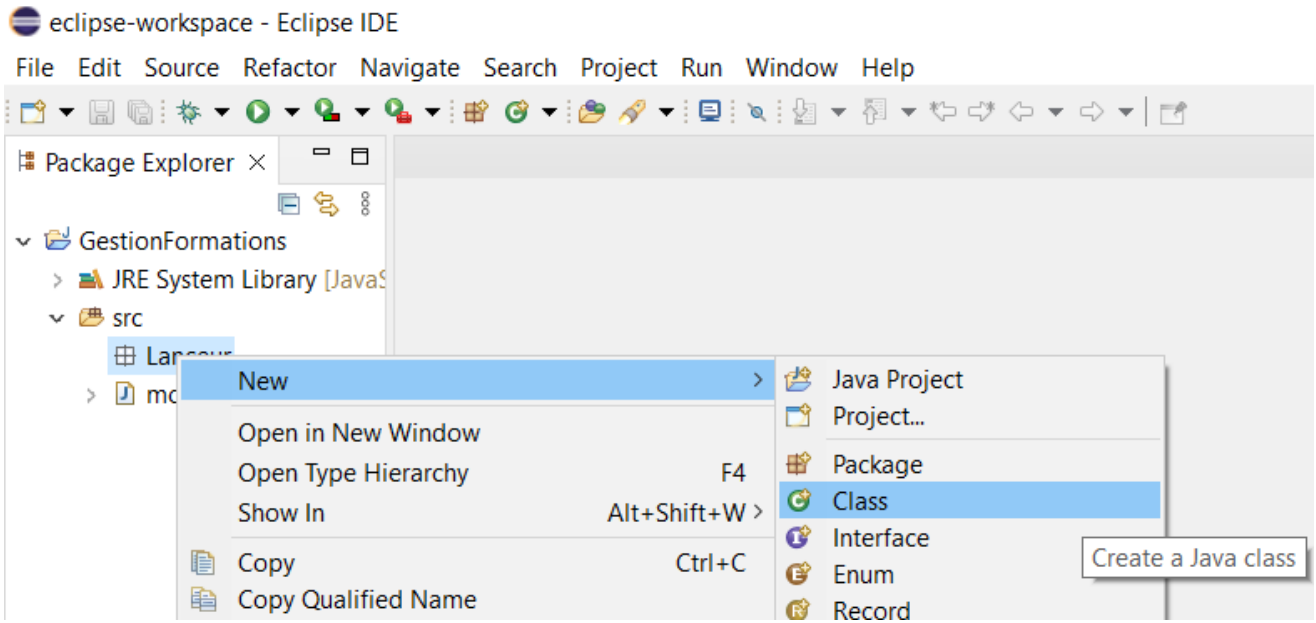


IV. Environnement de travail Java

Création projet Java

- Eclipse crée alors un package
- Passant désormais à la création d'une classe principale en cliquant droit sur le package (lanceur) , puis new/class





IV. Environnement de travail Java

Création projet Java

- Ecrire le nom de la classe , ici ‘Launcher‘

eclipse-workspace - GestionFormations/src/Lanceur/Launcher.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- GestionFormations
  - JRE System Library [JavaS
  - src

Launcher.java

```
1  package Lanceur;
2
3  public class Launcher {
4
5      public static void main(String[] args) {
6          // TODO Auto-generated method stub
7          System.out.println("Bonjour LPR2SI");
8
9      }
10
11 }
12
```

Problems Javadoc Declaration Console

```
<terminated> Launcher [Java Application] C:\Users\LENOVO\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.6.v20230125-1136\jre\bin\javaw.exe (3
Bonjour LPR2SI
```

Atelier I :

- Créer un Nouveau projet Hello World sous Eclipse
- Compiler le projet
- Essayer de vous familiariser avec les principales commandes d'Eclipse

V. Syntaxe du langage Java

Commentaires dans le code source

- Trois façons d'inclure des commentaires :
  - Tout texte entre « // » et la fin de la ligne

    // Commentaires sur une seule ligne
  - Tout texte entre « /* » et « */ »

    / Commentaires*
    sur un nombre important voire très important
    *de lignes */*
  - Les textes entre « /** » et « */ » sont utilisés pour créer des commentaires que l'exécutable JAVADOC pourra traiter afin de produire une documentation (cf. documentation de l'API Java)

    */** Commentaires destinés*
    *à la documentation */*

V. Syntaxe du langage Java

Identificateurs

- Un identificateur permet de nommer les variables, méthodes, classes, ...
- Il comporte au moins un caractère
- Il est formé de majuscules, minuscules, chiffres, $, _, caractères accentués
- Le premier caractère ne doit pas être un chiffre
- JAVA est sensible à la casse : monde et mOnde sont deux identificateurs distincts.

V. Syntaxe du langage Java

Mots-clé

abstract	double	int	strictfp
boolean	else	interface	super
break	extends	long	switch
byte	final	native	synchronized
case	finally	new	this
catch	float	package	throw
char	for	private	throws
class	goto	protected	transient
const	if	public	try
continue	implements	return	void
default	import	short	volatile
do	instanceof	static	while

V. Syntaxe du langage Java

Typage fort

- Java est un langage à typage fort :
  - Chaque variable a un type
  - La valeur doit être du même type que la variable
  - On ne peut (normalement) pas mélanger des variables de type différent
- Pourquoi?
  - Utilisation efficace de la mémoire
  - Vérification de la logique du programme
- Les règles pour entier et réel sont plus souples :
  - Transtypage automatique : entier → réel
  - Transtypage explicite (explicit **cast**) : réel → entire
    - int i = **(int)**1.7; //Valeur tronquée, i vaut 1

V. Syntaxe du langage Java

Les types de base

- En Java, tout est objet sauf les types de base.
- Il y a huit types de base :
  - Un type booléen pour représenter les variables qui ne pouvent prendre que 2 valeurs (vrai/faux, 0/1, etc) : **boolean** avec les valeurs associées true et false
  - Un type pour représenter les caractères : **char**
  - Quatre types pour représenter les entiers de divers taille : **byte**, **short**, **int** et **long**
  - Deux types pour représenter les réels : **float** et **double**

V. Syntaxe du langage Java

Les type de base

Type de base	Type	Nombre de bits	Valeurs possible
boolean	booléen	De 1 à 32(Non défini par la specification)	true et false
byte	entier	8	signées
short	entier	16	signées
int	entier	32	signées
long	entier	64	signées
float	virgule flottante	32	IEEE 754
double	virgule flottante	64	IEEE 754
char	caractère	16	Unicode

V. Syntaxe du langage Java

Le type String

- Une chaîne de caractères est déclarée avec le mot-clé « **String** »
- Ce n’est pas un type de base
- Il se manipule comme un type de base
- Ce n’est pas équivalent à un tableaux de caractères
- **String** s = "Bonjour tous le monde!”;
- L‘operateur **«+** » est utilisé pour concaténer les chaînes de caractères

Les types de base : Déclaration/Initialisation

- Considérons l'instruction suivante :

  int i;

- C'est ce qu'on appelle une déclaration d'une variable de type entier.
- A l'exécution de cette instruction, java réserve un espace mémoire de 32 bits, permettant de stocker un entier.
- Cet espace mémoire correspond uniquement à la variable i.
- Si une variable n'est pas initialisée après sa déclaration et que l'on tente de l'utiliser (par exemple pour l'afficher ou effectuer une opération), une erreur de compilation se produira. Java, plus strict que d'autres langages comme le C ou le C++, interdit l'utilisation d'une variable qu'il considère comme non initialisée, afin d'éviter tout comportement imprévisible du programme.

Les types de base : Déclaration/Initialisation

Initialisation:

- Il y a deux façons d'initialisation:
  - On peut initialiser lors de la déclaration en utilisant la syntaxe suivante :

```
int i=2;
```

  - On peut initialiser plus tard en utilisant une affectation :

```
int i;
i=2;
```

Les types de base : Déclaration/Initialisation

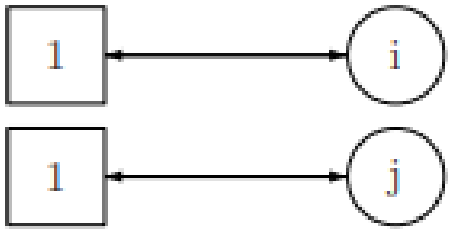
Affectation:

- L'affectation des types primitifs en java fonctionne par valeur. Voyons ce que cela signifie sur un petit exemple.

```
int i = 1;    // (1)
int j;        // (2)
j = i;        // (3)
```

Ce bout de code a le comportement suivant :

1. Création et initialisation d'une variable i : un espace mémoire de 32 bits est associe a i. Dans cet espace mémoire est écrit un encodage de l'entier « 1 ».
2. Création d'une variable j : un espace mémoire de 32 bits est associé à j.
3. Affectation de la valeur de i à j.



Après l'exécution de ce code, il n'y a donc plus aucun lien entre i et j.

V. Syntaxe du langage Java

Typage dynamique

- Depuis Java 10, on peut déclarer des variables avec inférence de type à l'aide du mot-clé **var**. Le type est déterminé automatiquement par le compilateur, mais reste statique (contrairement au typage dynamique de Python).

```
var a = 5;
```

  - Une variable déclarée avec le mot-clé **var** doit être initialisée
  - Le type de la variable ***a*** est déterminé en fonction de sa valeur initiale
  - Elle ne peut pas changer de type

Modifiable/non modifiable

- La nature **modifiable** ou **non modifiable** d'une donnée peut être définie lors de la déclaration par l'indication du mot réservé **final**.
- Elle ne pourra donc plus être modifiée par le programme (toute tentative de modification produira un message d'erreur lors de la compilation).
- Exemples :
  - **final double** PI = 3.141592653;
  - **final int** MAX_SIZE=15 ;

Arithmétique et opérateurs

- Quelle est la valeur de : 5+3*4+(12/4+1)
- Règles de précédence :

Niveau	Symbole	Signification
1	()	Parenthèse
2	* / %	Produit Division Modulo
3	+ -	Addition ou concaténation Soustraction

Arithmétique et opérateurs

++nomVar	pré-incrémentation
nomVar++	post-incrémentation
--nomVar	pré-décrémentation
nomVar--	post-décrémentation

```
int i = 0;
int a = i++;
// affecte la valeur de 'i' à 'a'
// puis incrémente 'i' de 1
int b = ++i;
// incrémente 'i' de 1
// et affecte la valeur de 'i' à 'b'
```

Arithmétique et opérateurs

- Pour comparer deux valeurs:

Opérateur	Exemple	Renvoie TRUE si
> >=	v1 > v2 v1 >= v2	v1 plus grand que v2 Plus grand ou égal
< <=	v1 < v2 v1 <= v2	Plus petit que Plus petit ou égal à
== !=	v1 == v2 v1 != v2	égal différent

- Opérateurs logiques:

Opérateur	Usage	Renvoie TRUE si
&&	expr1 && expr2	expr1 et expr2 sont vraies
||	expr1 || expr2	Expr1 ou expr2, ou les deux sont vraies
!=	expr1 != expr2	si expr1 est différent de expr2

Arithmétique et opérateurs

Exemples :

```
boolean x;

x= true;

x= false;

x = (5==5); /* L'expression (5==5) est évaluée et la valeur retournée est affectée à x qui
               vaut alors VRAI*/

x = (5!=4); // x vaut VRAI, 5 est différent de 4

x = (5>5);  // x vaut FAUX, 5 n'est pas supérieur strictement à 5

x = (5<=5); // x vaut VRAI, 5 est bien inférieur ou égal à 5
```

Arithmétique et opérateurs

Exemples :

```
boolean a, b, c;
a= true;
b= false;
c= (a && b); // c vaut FAUX
c= (a || b); // c vaut VRAI
c= !(a && b); // c vaut VRAI
c=!a; // c vaut FAUX
```

Arithmétique et opérateurs

- Il existe des opérateurs d'affectation qui réalisent à la fois :
  - Une opération arithmétique ou logique
  - Et l'affectation proprement dite

Opérateur	Exemple	Équivalent à
+=	expr1 += expr2	expr1 = expr1 + expr2
-=	expr1 -= expr2	expr1 = expr1 – expr2
*=	expr1 *= expr2	expr1 = expr1 * expr2
/=	expr1 /= expr2	expr1 = expr1 / expr2
%=	expr1 %= expr2	expr1 = expr1 % expr2

V. Syntaxe du langage Java

Instruction d’affichage

- Sous Java pour afficher un message sur la console on utilise l'une des deux méthodes suivantes :

  System.out.print(“Votre message”);

  System.out.println(“Votre message”);

- La différence entre ces deux méthodes (print et println) c'est que println affiche le message et retourne à la ligne tandis que print affiche le message est reste sur la même ligne.

Instruction d'affichage

```
7  public class testClasse {
8
9      public static void main(String[] args) {
10
11         System.out.println("Mon premier Message.");
12         System.out.println("Mon deuxième Message.");
13     }
```

Problems @ Javadoc Declaration Console

```
<terminated> testClasse [Java Application] /usr/lib/jvm/java-11-openjdk-amd64/bin/java (22 nov. 2019 à 20:09:41)
Mon premier Message
Mon deuxième Message
```

```
7  public class testClasse {
8
9      public static void main(String[] args) {
10
11         System.out.print("Mon premier Message.");
12         System.out.print("Mon deuxième Message.");
13     }
```

Problems @ Javadoc Declaration Console

```
<terminated> testClasse [Java Application] /usr/lib/jvm/java-11-openjdk-amd64/bin/java (2
Mon premier Message.Mon deuxième Message.
```

Instruction d'affichage

- La méthode print (ou println) peuvent aussi afficher les valeurs des variables. Par exemple, le code suivant affiche la valeur de la variable i.

```
public static void main(String[] args) {

    int i=3;
    System.out.println(i);

}
```

- On peut aussi concaténer l'affichage en ajoutant du texte à nos variables

```
8
9⊖    public static void main(String[] args) {
10        int i=3;
11        System.out.println("le contenue de i est egale a "+ i);
12    }
```

Problems @ Javadoc Declaration Console

```
<terminated> testClasse [Java Application] /usr/lib/jvm/java-11-openjdk-amd64/bin/java (22 nc
le contenue de i est egale a 3
```

Instruction de lecture

Classe java.util.Scanner

- Par exemple, ce code lit deux chiffres de la console et imprime leur somme

```
Scanner in = new Scanner(System.in);
int i = in.nextInt();
int j = in.nextInt();
System.out.println(i+j);
```

- Le Scanner fournit également des méthodes pour tester s'il reste des entrées et, le cas échéant, quel type de jeton apparaît ensuite.
- Cette fonctionnalité est fournie par des méthodes comme hasNextInt, hasNextDouble.
- Par exemple, le code suivant lit les entiers et les additionne jusqu'à ce qu'il n'y ait plus d'entrée ou qu'un jeton non numérique soit rencontré :

```
Scanner in = new Scanner(System.in);
int sum = 0;
while (in.hasNextInt()) {
  sum += in.nextInt();
}
```

Instruction de lecture

Classe java.util.Scanner

- Les méthodes de Scanner :

Méthode	Fonction
nextBoolean()	lit et convertit le prochain jeton en une valeur booléenne
nextInt()	lit et convertit le prochain jeton en une valeur entière
nextLong()	lit et convertit le prochain jeton en une valeur de type long
nextDouble()	lit et convertit le prochain jeton en un nombre à virgule (double)
nextString() ou next()	lit le prochain jeton et le renvoie sous forme de chaîne de caractères (String)
nextLine()	lit jusqu'au prochain retour à la ligne et renvoie une chaîne de caractères (String)

Instruction de lecture

Classe java.util.Scanner

- Les méthodes de Scanner :

Méthode	Fonction
hasNextBoolean()	renvoie true si le prochain jeton est soit true soit false
hasNextInt()	renvoie true si le prochain jeton est un entier
hasNextLong()	renvoie true si le prochain jeton est un long
hasNextDouble()	renvoie true si le prochain jeton est un nombre réel
hasNextString() ou hasNext()	renvoie true s'il reste au moins un jeton à lire
hasNextLine()	renvoie true s'il y a une autre ligne d'entrée à lire

Atelier III :

Ecrire un programme qui demande l'âge et la taille de l'utilisateur, puis les affiche sous la forme suivante :

Vous avez *age_utilisateur* ans et vous mesurez *taille_utilisateur* m

Les structures de contrôle

- Les structures de contrôle sont des constructions de programmation qui permettent de contrôler le flux d'exécution d'un programme en fonction de certaines conditions. Les structures de contrôle les plus courantes sont :
  - La structure conditionnelle
  - La structure itérative
- Ces structures permettent d'ajouter de la logique et de la complexité à un programme, en permettant de prendre des décisions en fonction de certaines conditions et d'exécuter certaines actions plusieurs fois.

Les structures de contrôle

La structure conditionnelle : if – else

- La structure if est la structure conditionnelle la plus courante en Java. Elle permet de faire un choix en fonction d'une condition.

  if (condition) instruction;

  Ou

  if (condition) { bloc d'instructions}

- Il est possible d'ajouter une clause else pour exécuter un code si la condition est fausse

  if (condition) instruction1; else instruction2;

  Ou

  if (condition) { 1er bloc d'instructions} else {2ème bloc d'instruction}

- ***N.B*** *: La condition doit être un booléen ou renvoyer une valeur booléenne*

Les structures de contrôle

La structure conditionnelle : SWITCH – CASE

- L'utilisation de if / else peut s'avérer lourde quand on doit traiter plusieurs sélections et de multiples alternatives
- Pour cela il existe en Java le switch / case
- Tout comme l'instruction if, le switch permet de déclencher des traitements en fonction de la valeur d'une expression.
- L'exécution des instructions correspondant à une alternative commence au niveau du *case* correspondant et se termine à la rencontre d'une instruction break ou arrivée à la fin du switch :

```
Int number
....
switch (number)
{                    case 1 : instructions;  break;
                     case 2 : instructions;      break;
                     default : instructions;
}
```

Les structures de contrôle

La structure itérative : Boucle indéterminée (while)

- On veut répéter une ou plusieurs instructions un nombre indéterminés de fois : on répète l'instruction ou le bloc d'instruction tant qu'une certaine condition reste vraie
- Nous avons en Java la boucle **while** (tant que)

while (condition)

{bloc d'instructions}

  - les instructions dans le bloc sont répétées tant que la condition reste vraie.
  - On ne rentre jamais dans la boucle si la condition est fausse dès le départ

Les structures de contrôle

La structure itérative : Boucle indéterminée (do..while)

- Un autre type de boucle avec le **while**:

```
do
        {bloc d'instructions}
while (condition)
```

  - les instructions dans le bloc sont répétées tant que la condition est satisfaite.
  - On rentre toujours au moins une fois dans la boucle : la condition est testée en fin de boucle.

Les structures de contrôle

Les instructions de saut : BREAK / CONTINUE

- break: achève immédiatement la boucle
- continue: ignore le reste des instructions et recommence au début de la boucle

```
for (int i=0; i<10 ;i++){
    if (i==5) continue; // Si i=5, on recommence au début
    if (i==7) break;    /*Si i=7, on sort de la boucle et
                        les instructions suivantes sont
                        exécutées */
    System.out.println("La valeur de i est : " + i);
}
```

Merci de votre attention

Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java/Python

Pr. *Ilyass OUAZZANI TAYBI*

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026